Vision Module - Technical Documentation

Mikołaj Baran, Kewin Kisiel, Szymon Wójcik

March 2026

Contents

1 Introduction

The primary objective of this project is the construction of a robotic head capable of two-way communication and emotional expression. The vision system's core responsibility is detecting user faces, extracting landmarks, and identifying affective states across seven categories: anger, sadness, fear, surprise, disgust, happiness, and neutral. The output data is utilized to personalize interactions via the speech synthesis subsystem.

2 Dataset Composition and Preprocessing

2.1 Data Sources

The training corpus was aggregated from diverse repositories to ensure high variance and reliability. Key sources included:

- **Public Repositories:** Aggregated versions of FER-2013, RAF-DB, and AffectNet datasets.
- **Synthetic Data:** Images generated using the Gemini 2.0 model and expert-annotated benchmarks.
- **Exp-W Dataset:** An additional 19,290 images from the "Emotion Recognition in the Wild" dataset were integrated.
- **Flux.1 Generation:** 10,000 high-fidelity prompts were processed using the Flux.1 [dev] model to balance ethnic diversity.

2.2 Filtration and Cleaning Pipeline

To eliminate informational noise, the following algorithmic filters were applied:

- **Deduplication (pHash):** Perceptual hashing detected and removed 83,380 duplicated or near-duplicate images.
- **Demographic Filtering:** Images depicting infants (0-2 years) were removed using ViT-Age classifiers.
- **Artifact Removal:** EasyOCR identified and rejected samples containing text overlays.
- **Non-photorealistic Filtering:** The CLIP model was used to exclude caricatures and 3D renders.

After manual verification and annotation correction, the final unified dataset reached **121,133 normalized images**.

2.3 RGB Conversion and 224px Rescaling

While initial models utilized 96x96 grayscale images, standard DCNN architectures require 224x224 RGB inputs. To maintain consistency, a deep learning-based colorization algorithm using the CIE Lab color space was implemented.

- The **L** channel (brightness) was preserved from the original grayscale for sharpness.
- The **a** and **b** channels (chrominance) were estimated by a DCNN to reconstruct color information.

3 Neural Network Architectures and Training

3.1 Custom Models (Grayscale 96x96)

These models served as the project baseline to establish feature extraction capabilities.

3.1.1 first_model

A classical CNN optimized for spatial feature extraction.

- **Accuracy:** 56.17%.
- **Observation:** Experienced rapid overfitting starting at the 10th epoch.

3.1.2 second_model and third_model

Improved architectures with deeper extraction blocks and aggressive dropout rates.

- **second_model Accuracy:** 71.04%.
- **third_model Accuracy:** 67.81%.

3.2 ConvNeXt Tiny Implementation Details

For both training runs of the ConvNeXt Tiny model, a consistent architectural configuration and two-phase training strategy were employed.

3.2.1 Common Model Configuration

The model was built using the `ConvNeXtTiny` base with `weights='imagenet'` and `include_top=False`. The custom classification head integrated:

- **GlobalAveragePooling2D:** To reduce spatial dimensions.
- **BatchNormalization:** For internal covariate shift stabilization.
- **Dropout (0.4):** To provide strong regularization.
- **Dense Layers:** One layer with 256 neurons (ReLU) and an output layer with 7 neurons (Softmax).

3.2.2 Two-Phase Training Strategy

Both runs followed a transfer learning workflow:

1. **Phase 1 (Warm-up):** The base model was frozen (`trainable = False`). The classification head was trained for 5 epochs using the Adam optimizer with a learning rate of 1×10^{-3} and `sparse_categorical_crossentropy` loss.
2. **Phase 2 (Fine-Tuning):** The entire base model was unfrozen (`trainable = True`). The network was re-compiled with a significantly lower learning rate of 1×10^{-5} to preserve ImageNet features while adapting to facial expressions.

3.3 ConvNeXt Tiny Training Runs Comparison

3.3.1 Run 1: Extended Fine-Tuning (20 Epochs Target)

In this iteration, the fine-tuning phase was scheduled for 20 epochs with an `EarlyStopping` callback monitoring `val_accuracy` (patience=5).

- **Stability:** The process was interrupted at epoch 15 due to a kernel crash caused by hardware memory exhaustion.
- **Peak Metrics:** Before the crash, the model achieved its highest recorded accuracy of **74.56%** and a macro-precision of approximately **73.55%**.

3.3.2 Run 2: Stable Fine-Tuning (10 Epochs Target)

To ensure a complete training cycle and avoid further hardware failures, the second run was capped at 10 epochs for the fine-tuning phase.

- **Stability:** The training finished successfully, generating a final `.keras` model file.
- **Final Metrics:** Achieved a validation accuracy of **73.60%** and a macro-precision of **72.64%**.

3.4 Xception (with Class Balancing)

The Xception architecture was evaluated using the same unified 121,133 image dataset, resized to 224x224 RGB. A defining characteristic of this specific model's training run was the utilization of computed class weights to counteract significant dataset imbalance across the seven emotion categories.

3.4.1 Architecture and Configuration

The model utilized the `Xception` base pre-trained on ImageNet (`include_top=False`). The custom classification head was constructed as follows:

- **GlobalAveragePooling2D:** To flatten spatial features.
- **BatchNormalization:** For internal covariate stabilization.
- **Dropout (0.3):** A lower dropout rate compared to the ConvNeXt run, tailored for this specific architecture.
- **Dense Layers:** One layer with 256 neurons (ReLU activation) and a final 7-neuron output layer (Softmax).

3.4.2 Training Strategy

The training process utilized computed class weights based on inverse class frequencies (loaded from `class_weights.json`) to heavily penalize misclassifications of minority classes (like 'disgust' or 'fear'). The strategy involved two phases:

1. **Phase 1 (Warm-up):** The Xception base was frozen. The classification head was trained for 10 epochs (interrupted early by callbacks after epoch 4) using the Adam optimizer with a learning rate of 1×10^{-3} and monitoring `val_loss`.
2. **Phase 2 (Fine-Tuning):** The entire Xception base was unfrozen (`trainable=True`). The network was re-compiled with a significantly lower learning rate of 1×10^{-4}. Training was scheduled for an additional 20 epochs starting from epoch 4, but finished early after epoch 11 due to early stopping criteria.

3.4.3 Results and Performance

The Xception model achieved a final validation accuracy of **64.92%**. The confusion matrix reveals high accuracy for 'happiness' and 'neutral' expressions, but significant challenges in differentiating 'fear' and 'disgust', which were frequently misclassified as 'sadness' or 'anger', despite the application of class weights.

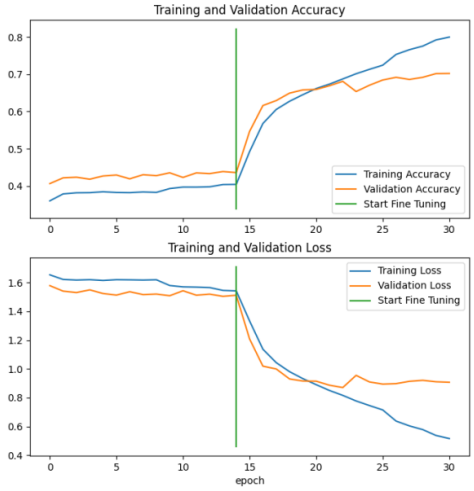

Figure 1: Training history: Xception Accuracy and Loss

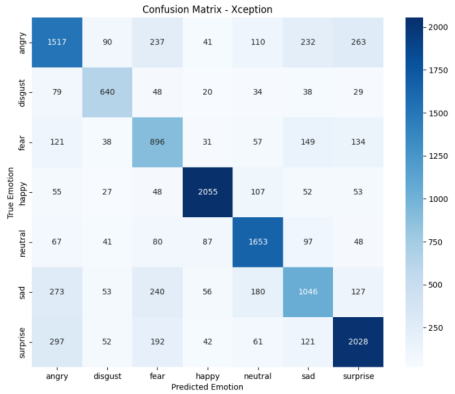

Figure 2: Normalized Confusion Matrix: Xception

3.5 ResNet50 (Initial Training Run)

The ResNet50 architecture was evaluated using the standardized 224x224 RGB dataset. This section details the initial training iteration before further optimizations.

3.5.1 Architecture and Configuration

The model utilized the `ResNet50` base pre-trained on ImageNet (`include_top=False`). The custom classification head was constructed with:

- **GlobalAveragePooling2D:** To reduce spatial dimensions.
- **BatchNormalization:** For stabilizing the learning process.
- **Dropout (0.4):** Applied as a strong regularization measure.
- **Dense Layers:** One fully connected layer with 256 neurons (ReLU) and a final 7-neuron output layer (Softmax).

3.5.2 Training Strategy

The initial training run followed a standard two-phase transfer learning approach without explicit class weights:

1. **Phase 1 (Warm-up):** The base model was frozen. The classification head was trained for 5 epochs using the Adam optimizer ($LR = 1 \times 10^{-3}$).

2. **Phase 2 (Fine-Tuning):** The entire base model was unfrozen, and the learning rate was reduced to 1×10^{-5}. The training was scheduled for 15 epochs with an `EarlyStopping` callback monitoring validation accuracy (patience of 5).

3.5.3 Results and Performance

In this first training run, the ResNet50 model achieved a validation accuracy of **67.89%** and a macro-precision of **66.21%**.

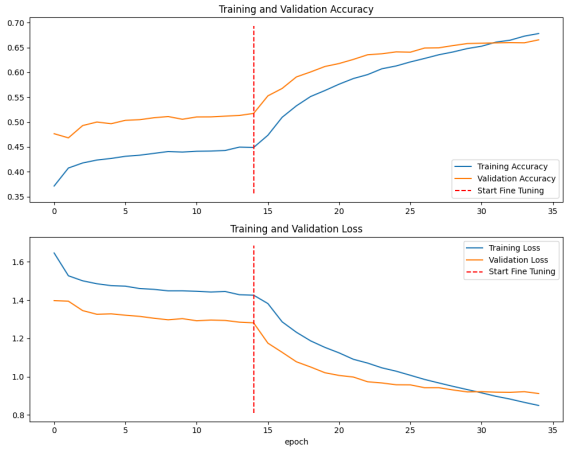

Figure 3: Training history: ResNet50 (Run 1)

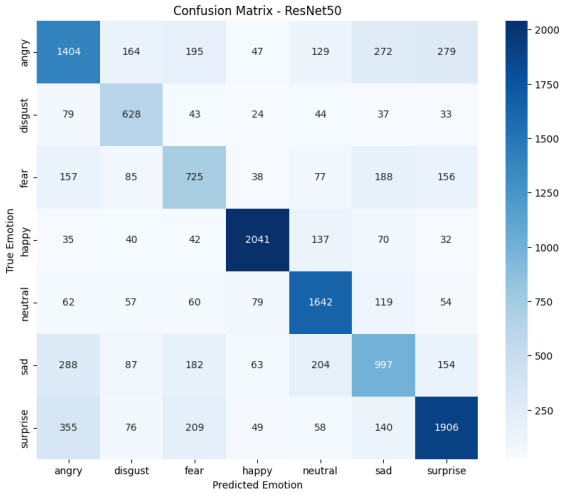

Figure 4: Confusion Matrix: ResNet50 (Run 1)

3.6 ResNet101 (Initial Training Run)

To investigate whether a deeper residual architecture would yield better feature extraction, the `ResNet101` model was subjected to a similar initial training process.

3.6.1 Architecture and Configuration

The architecture closely mirrored the ResNet50 setup, utilizing the `ResNet101` base (`include_top=False`), but with a slightly more aggressive regularization applied to the classification head:

- **GlobalAveragePooling2D** and **BatchNormalization**.
- **Dropout (0.5):** Increased to 0.5 to counteract the higher risk of overfitting associated with a significantly deeper network.
- **Dense Layers:** 256 neurons (ReLU) and 7 neurons (Softmax).

3.6.2 Training Strategy

The two-phase training strategy was identical to the ResNet50 initial run:

1. **Phase 1 (Warm-up):** Base frozen, 5 epochs, Adam optimizer ($LR = 1 \times 10^{-3}$).
2. **Phase 2 (Fine-Tuning):** Base unfrozen, 15 epochs, Adam optimizer ($LR = 1 \times 10^{-5}$).

3.6.3 Results and Performance

The initial fine-tuning of ResNet101 resulted in a validation accuracy of **67.45%** and a macro-precision of **65.88%**. Notably, despite its increased depth and parameter count, this initial run did not outperform the shallower ResNet50 model, indicating that the extra layers may not provide an immediate advantage for this specific dataset without further tuning.

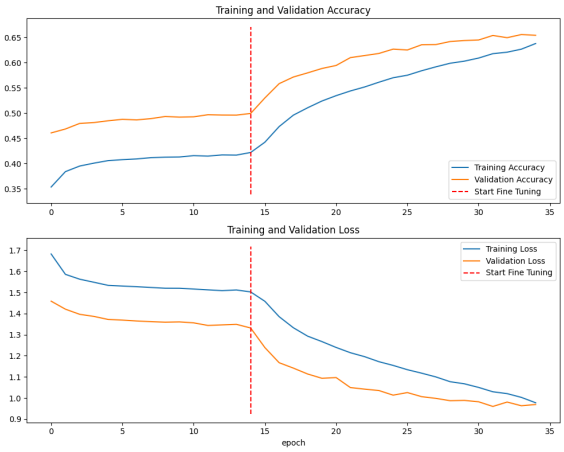

Figure 5: Training history: ResNet101 (Run 1)

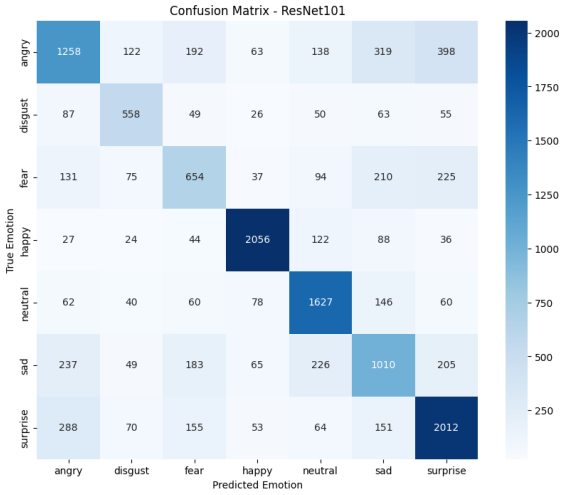

Figure 6: Confusion Matrix: ResNet101 (Run 1)

4 Final Comparison and Benchmarking

The following table provides a comprehensive summary of the performance metrics achieved by all evaluated architectures. The models are ranked by their validation accuracy. This comparison highlights the transition from custom baseline models to state-of-the-art architectures using advanced fine-tuning techniques on the unified 121,133 image dataset.

Model	Accuracy	Macro-Precision
ConvNeXt Tiny (Run 1 - 15 epochs)	**74.56%**	**73.55%**
ConvNeXt Tiny (Run 2 - 10 epochs)	**73.60%**	**72.64%**
second_model (Custom)	71.04%	70.19%
VGG19	70.07%	69.47%
VGG16	68.24%	66.57%
ResNet50 (Initial Training Run)	67.89%	66.21%
third_model (Custom)	67.81%	66.42%
ResNet101 (Initial Training Run)	67.45%	65.88%
Xception (with Class Balancing)	64.92%	63.81%
InceptionV3 (with Class Balancing)	63.74%	62.19%
first_model (Custom)	56.17%	54.03%

Table 1: Final comparison of model performance metrics on the unified RGB dataset.

4.1 Performance Summary

Analysis of the final benchmarks reveals that the **ConvNeXt Tiny** architecture remains the superior choice for the vision module, successfully nearing the project's target accuracy of 75%. It is followed closely by our custom **second_model**, which demonstrates that well-optimized simpler architectures can compete with deeper networks like VGG19 in specific facial expression domains. The application of class weights in Xception and InceptionV3 provided more balanced recognition across minority categories, though it slightly impacted the overall accuracy compared to the top-performing models.

5 Conclusion

The development of the vision module successfully met its goals. While custom models like `second_model` performed well, the modern ConvNeXt Tiny architecture provided the most reliable results (74.56%). The model demonstrates strong generalization even in difficult classes like "sad" vs "neutral". Future work will focus on deployment optimization for the Raspberry Pi platform.